



Security Review Report for Axelion

May 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
 - Security Review Lead
 - Scope
 - Changelog
4. Severity Structure
 - Severity characteristics
 - Issue symbolic codes
5. Findings Summary
6. Weaknesses
 - Non-Standard 2 Decimals May Break Integrations
 - Frozen accounts cannot be burned through operatorBurn

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This report covers the security review for the Philippine Peso Stablecoin, developed by Axelion.

Our security assessment was a full review of the code, spanning a total of 1 day.

During our review, we not identify any major severity vulnerabilities.

We did identify a few minor severity vulnerabilities.

All reported issues were fixed by the development team and subsequently verified by us.

We can confidently say that the overall security and code quality have increased after completion of our audit.

3. Security Review Details

- Review Led by

Jahyun Koo, Lead Security Researcher

- Scope

The analyzed resources are located on:

<https://github.com/PaTRiiiCK15/phpa-smart-contract/tree/10a02f215e687129eb03906ce17bc20a4f7112a9>

The issues described in this report were fixed in the following commit:

Commit Hash: [0fbb587cf55f1cb75ca4874df6e6e4eef448a1b2](#)

- Changelog

■ 18 May 2026	Audit start
■ 19 May 2026	Initial report
■ 11 June 2026	Revision received
■ 12 June 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

- 1. Impact of the vulnerability
- 2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

▪ Severity Characteristics

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical	Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.
High	Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium

Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.

Low

Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.

Informational

Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

▪ Issue Symbolic Codes

Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity

Number of findings

Critical	0
High	0
Medium	0
Low	1
Informational	1

Total:

2



Low

Informational



Fixed

6. Weaknesses

This section contains the list of discovered weaknesses.

AXE1-1 | Non-Standard 2 Decimals May Break Integrations

Fixed 

Severity:

Low

Probability:

Unlikely

Impact:

Low

Description:

The token decimals is set to **2**:

```
function decimals() public pure override returns (uint8) {  
    return 2;  
}
```

Although valid under ERC-20, most EVM tokens and integrations commonly use **6**, **8**, or **18** decimals. Using **2** decimals may create compatibility issues with exchanges, wallets, bridges, and DeFi protocols that assume more common precision standards.

For example, USD Coin (USDC) uses 6 decimals, even though the U.S. dollar itself only has 2 decimals, similar to the Philippine Peso.

Impact

- Integration issues with exchanges and third-party services
- Incorrect rounding or accounting assumptions
- Reduced DeFi and cross-chain composability
- Limited precision for fees or calculations

Remediation:

Use a more standard decimal format such as **6**, **8**, or **18**:

```
function decimals() public pure override returns (uint8) {  
    return 6; // or 8 / 18  
}
```

AXE1-2 | Frozen accounts cannot be burned through operatorBurn

Fixed ✓

Severity:

Informational

Probability:

Rare

Impact:

Informational

Description:

The `_update` hook rejects any transfer whose sender is marked as frozen, and this check is also applied to burn operations because `_burn` internally calls `_update(from, address(0), amount)`. As a consequence, once an address has been frozen via `freeze()`, the operator can no longer reduce its balance through `operatorBurn`.

```
function _update(
    address from,
    address to,
    uint256 amount
) internal override(ERC20, ERC20Pausable) {
    super._update(from, to, amount);

    if (from != address(0)) {
        require(!_frozen[from], "PHPA: sender address is frozen");
    }
    if (to != address(0)) {
        require(!_frozen[to], "PHPA: recipient address is frozen");
    }
}
```

Under the current implementation, the operator must unfreeze the address before `operatorBurn` can succeed, which briefly re-exposes the balance to outbound transfers initiated by the holder.

Remediation:

- Let `operatorBurn` bypass the freeze check in `_update`, for example by treating the burn path (`to == address(0)`) as exempt when the caller holds `BURNER_ROLE`.
- Apply the same exemption to the pause check if seizing balances during an emergency pause is part of the intended operational model.

hexens x  Axelion

